

Build a Large Language Model from Scratch

This thesis is a summary of [1] and reports on the experience with it.

All code implementations developed for this report are available in the GitHub repository [2].

REPORT 5

Pretraining on Unlabeled Data

Giorgi Gogsadze

Revaz Gogvadze

Supervised by Prof. Walter Tichy

Structure

1. Evaluating Generative Text Models
2. Training and Validation Losses
3. LLM Training Loop Implementation
4. Improving the Training Loop
5. Decoding Strategies (temperature, top-k)
6. Saving and Loading Model Weights
7. Loading Pretrained GPT-2 Weights

Except for the exercise solutions and tests, all the code presented in this work are adapted from Chapter 5 and Appendix D of [1].

The goal of the report is to pretrain the model on unlabeled (raw) text data, that will allow our model to learn patterns of language. Unlike supervised learning tasks, language models do not require labeled datasets. Instead, they are trained to predict the next token in a sequence. By repeatedly performing this prediction across large text, the model gradually learns grammar, semantic relationships, and contextual dependencies between words. But before we train the model, we need a way to measure how well it performs.

Evaluating Generative Text Models

In language models, performance is measured by how accurately the model predicts the next token in a sequence. When the model gives high probability to the correct token, the prediction is good. If the probability is small, the prediction is poor.

The difference between the probability of token predicted by model and the actual token is called a **loss**. Smaller loss means better predictions.

During training the model tries to minimize this loss by tuning the parameters, which are initially generated randomly (or imported from an existing model). Hence, the loss value serves as an indicator of training progress, and a well-behaving model should show decreasing loss values over time.

To see if the model is learning general language patterns rather than memorizing the data, we evaluate the performance on 2 datasets:

- **Training set** – used to update model parameters
- **Validation set** – used only for evaluation

If the training loss decreases, but the validation loss is high, the model is likely **overfitting**, i.e. it memorizes the training data instead of learning general patterns.

In language modeling, the correct output for each token is just the next token in the sequence. Hence, the training data is structured in a way that each input sequence is associated with a target sequence shifted by one position.

Example:

Input sequence	Target sequence
every effort moves	effort moves you

This means that the model learns to predict:

- given “every”, predict “effort”
- given “every effort”, predict “moves”
- given “every effort moves”, predict “you”

When repeating this task using large datasets, the model gradually learns patterns of language.

Calculating the loss

1. As discussed in the Report 4, when GPT model processed a sequence, the network outputs **logits**, which are numerical scores representing the model's confidence for each token in the vocabulary.

2. After softmax, we can look at the **probability** of all 50,257 tokens.

3. A good prediction is one that has a high probability for the right tokens. Ideally, the model should give a probability close to **1** for the correct token and close to **0** for others. But before training the model usually gives almost the same probability to every token in the vocabulary, so it acts like random guessing.

4. To evaluate the quality of the prediction, we compute the **logarithm of the probability** assigned to the correct token. Logarithm makes it easier to see how good the prediction is. When the correct token has high probability, the logarithm is close to 0. If the probability is very small, the logarithm is a large negative number, meaning the prediction is bad.

5. When training a language model, we are not just predicting a **single token**. Each training example contains many tokens. Instead of storing all these values separately, we compute their average. Thus we have one single number that indicates how well the model predicted the whole sequence.

6. Since probabilities are between **0 and 1**, their logarithms are always **negative or zero**. Multiplying the result by **-1** provides a positive value that can be minimized during training.

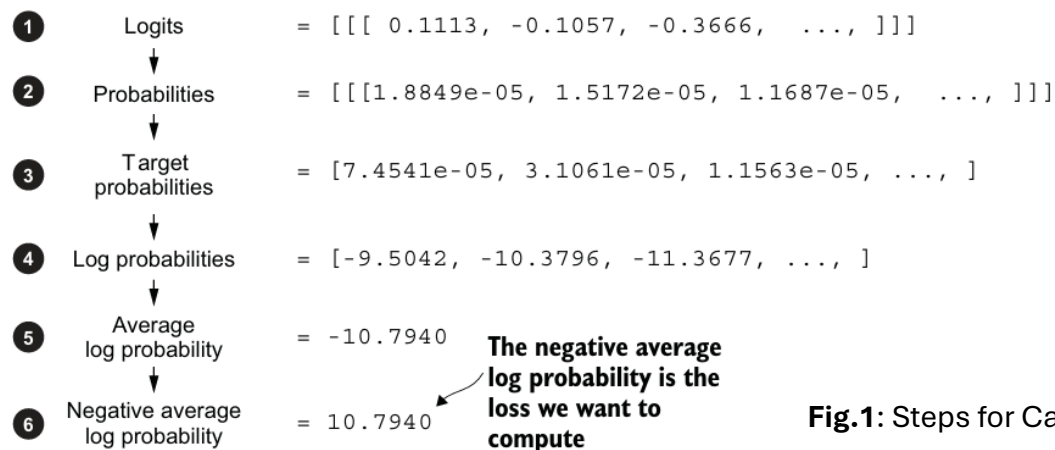


Fig.1: Steps for Calculating the loss (Source: [1], p. 137)

The resulting value indicates how far the predictions of the model are from the correct tokens. In practice, these steps are combined into a single metric called **cross-entropy loss**.

Role of Loss in Training

The loss value is central to training neural networks, because it tells us how wrong the predictions are. After the loss is calculated, the model performs **backpropagation**. This way it figures out how much each parameter in the network contributed to this error.

Backpropagation starts from the output layer and the algorithm calculates the change of the loss with respect to each of parameters. This gives a set of values called **gradients**. A gradient shows how to change a parameter to reduce the loss. If the gradient is large, the parameter has a strong impact on the error, otherwise, the parameter has little effect.

The optimizer then uses these gradients to update the model's parameters in the direction that minimizes the loss. By repeating this process for many training steps, the model gets better and better at prediction.

The training process can be summarized in 3 steps:

Forward pass – the model makes predictions.

Loss calculation – the error between predictions and the correct values is measured.

Backpropagation – the model determines which parameters caused the error and how they should be adjusted.

Training and Validation Losses

We illustrate the training process on a small dataset containing the short story “The Verdict” by Edith Wharton, so that the process can be done in a manageable setting, but also be enough to show the mechanics of language model training.

Preparing the Dataset

The dataset is loaded as raw text and converted into tokens using the tokenizer introduced in Report 2. The tokenized text is split into sequences matching the context length of the model (256 tokens in this implementation).

These sequences are batched together, so the model can process multiple examples at the same time, improving computational efficiency. The dataset is also divided into 90% training data and 10% validation data, in order to keep some data unseen during training.

Data Loaders

To efficiently feed batches of data to the model, the dataset is processed using **data loaders** (See Report 2). Each batch consists of two tensors: input tokens and target tokens. The target tokens are the input tokens shifted by one, so the model always learns to predict the next token in the sequence.

Computing Loss for the Dataset

To estimate the overall performance, the loss must be calculated across all batches in the dataset. This is done by iterating over the data loader and averaging the loss values.

The final **dataset loss** is the accuracy of the model in predicting tokens over the entire dataset.

Normally, the training and validation losses are both high before training starts, because the model parameters are randomly initialized. As training progresses, these values should gradually decrease.

Interpreting Training and Validation Loss

Both losses provide insight into the model's learning behavior. During successful training:

- **training loss decreases** - the model is learning patterns in the data
- **validation loss also decreases** - the model generalizes to unseen text

LLM Training Loop Implementation

After specifying how to calculate the training and validation loss, the next step is to train the language model. Training tunes the internal parameters to gradually improve its predictions and minimize the loss function.

The **training loop** is a standard procedure in deep learning and typically includes the steps shown in Figure 2:

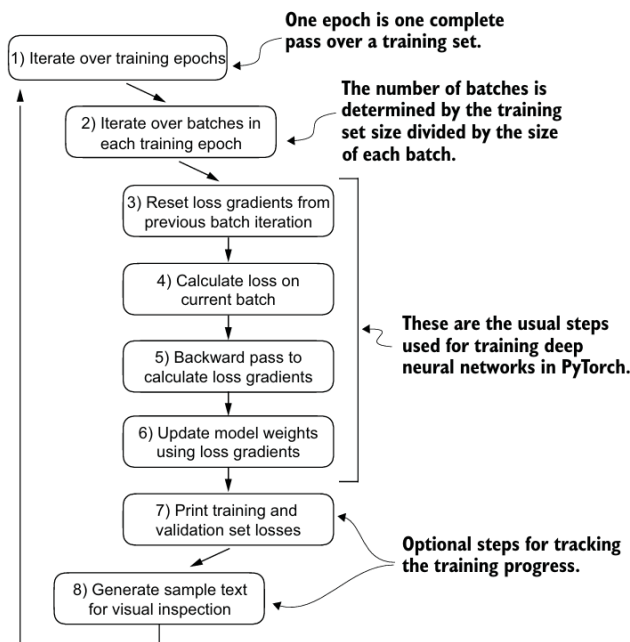


Fig.2: A typical training loop
(Source: [1], p. 146)

An **epoch** is one pass through the training data. Since datasets are generally large, it is needed to be split into smaller batches to it can be processed by the model.

At each iteration, the model receives a batch of token sequences and predicts the next token for each position. The predictions are then compared with the target tokens to compute the loss.

After it knows the loss, the model **backpropagates**, which results a set of gradients indicating how to change the parameters to decrease the loss. An optimization algorithm the uses these gradients to update the model weights. Over many iterations, these updates improve the model's ability to predict the next token.

Optimization with AdamW

For this implementation, we used **AdamW** optimizer, which is common standard for training transformer models.

A parameter w is updated as

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \text{gradient}$$

- w_{old} is the current value of a model parameter (weight).
- w_{new} denotes the updated parameter value.
- η is the learning rate, which controls the step size of the update.
- **gradient** represents the derivative of the loss function with respect to w_{old} , i.e.,

$$\text{gradient} = \nabla_w \mathcal{L}(w_{\text{old}})$$

But applying the same learning rate to all parameters can lead to slow training or unstability. Adam improves on this by automatically adapting the learning rate for each parameter, depending on how the gradients performed in prior training steps. This enables the model to learn faster and more reliably.

AdamW is a variant of Adam with **decoupled weight decay**. As explained in [3], weight decay slightly reduces the size of the model's weights during training:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \text{gradient} - \eta \lambda w_{\text{old}}$$

- λ is the weight decay coefficient (set to 0.1 in this implementation), controlling how aggressively weights are penalized for growing large.

This prevents the weights from becoming too large, which helps to avoid **overfitting** and helps the model to generalize better to new data.

Because of its stability, adaptive learning rates, and built-in regularization, AdamW has become a standard optimizer for training transformer-based models.

Training Results

While training, it is important to monitor the model's progress to ensure that learning occurs as expected. Training loss measures performance on the training data, while validation loss measures performance on unseen data.

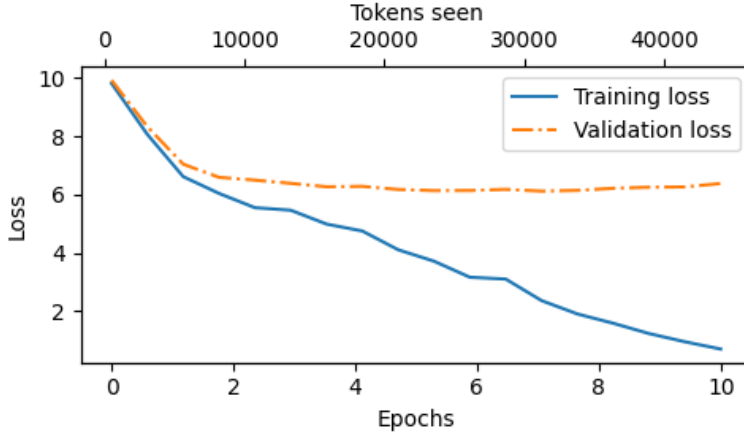


Fig.3: Simple Training Results

As we see in Figure 3, in the beginning of training both losses are very high and almost equal. Because the model weights are randomly initialized, it assigns almost same probabilities to all tokens.

$$p \approx \frac{1}{50257} \quad \rightarrow \quad -\log\left(\frac{1}{50257}\right) \approx 10$$

Our loss ≈ 9.9 is exactly what we expect.

In the early learning phase training loss is dropping quickly, as the model is learning basic patterns: common words, punctuation, simple token relationships. Even simple improvements drastically reduce loss.

In general Training loss keeps dropping strongly. Since the dataset is small, the model can almost remember the exact sequences. On the other hand, validation loss stops improving and even gets slightly worse. This is overfitting. However, this behavior is expected because the dataset contains only a few thousand tokens while the model has a much larger capacity.

In addition to numerical metrics, it is also useful to periodically generate text samples during training.

Epoch	Generated text
1	Every effort moves you,,,,,,,,,,,,,
5	Every effort moves you know the "Oh, and he had to me--I me. "Oh, I felt--and it's the "Oh, and I had been the donkey--and it to me, and down the "Oh,
10	Every effort moves you?" "Yes--quite insensible to the irony. She wanted him vindicated--and by me!" He laughed again, and threw back his head to look up at the sketch of the donkey. "There were days when I

At epoch 1, the model produces grammatically broken output, indicating no learned structure yet. By epoch 5, punctuation and short phrases emerge, but the model fixates on patterns like 'Oh,' suggesting it is latching onto frequent tokens. By epoch 10, the output closely mirrors actual passages from The Verdict — a clear sign of memorization rather than generalization, consistent with the diverging training/validation losses seen in Figure 3.

However, if the training set is small, the model might start to memorize the training data. In those situations, generated text may repeat passages from the dataset rather than generating new content.

That is why today's language models are typically trained on very large datasets that contain billions of tokens.

Improving the Training Loop

The training loop implemented earlier provides a simple example of how a language model can be pretrained. However, modern large language model training typically involves additional techniques that help improve training stability and performance.

1. Learning rate warmup
2. Cosine learning rate decay
3. Gradient clipping

Training can be unstable at the beginning if the learning rate is too high. This is solved by **Learning rate warmup** where it starts with a small learning rate and increasing it gradually up to the peak value.

```
lr_increment = (peak_lr - initial_lr) / warmup_steps
if global_step < warmup_steps:
    lr = initial_lr + global_step * lr_increment
else:
    lr = peak_lr
```

This allows the model to train carefully at the beginning, and avoids unstable updates at the early stage. Warmup usually takes up 0.1% to 20% of the total training steps.

After the warmup phase, the learning rate is decreased gradually via **cosine** decay. This helps to make larger updates early in training and smaller updates later, when it approaches a good solution. The learning rate can be computed as:

```
progress = ((global_step - warmup_steps) /
            (total_training_steps - warmup_steps))
lr = min_lr + (peak_lr - min_lr) * 0.5 * (1 + math.cos(math.pi * progress))
```

where *progress* indicates how far training has advanced.

This results in a smooth schedule where the learning rate first increases and then decreases smoothly (See figure 4). Cosine decay helps the model converge in a smoother way.

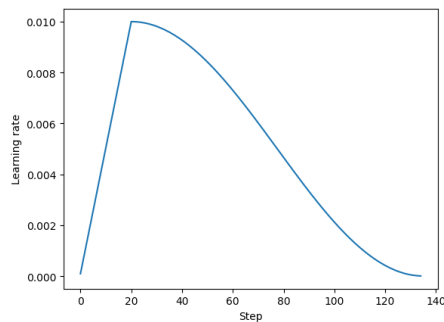


Fig.4: warmup followed by a cosine decay

Gradient Clipping

Training deep networks can also suffer from **exploding gradients**, where gradients become excessively large. **Gradient clipping** limits their magnitude to stabilize training.

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

If the gradient norm exceeds the threshold, the gradients are scaled down so that their magnitude stays within the specified limit. This stabilizes training, especially for deep transformer models.

Modified Training Function and Results

The training loop's overall structure is unchanged, but the learning rate is modified dynamically during training. These improvements are commonly used in modern transformer training pipelines and help achieve more stable and efficient optimization.

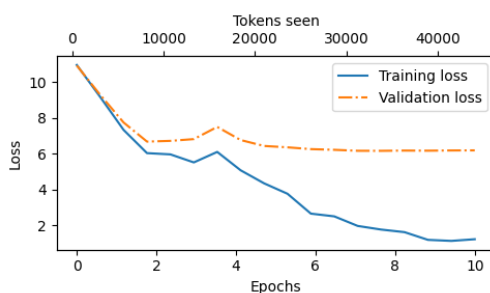


Fig.5: Improved Training Results

Due to hardware constraints, the model was trained on a relatively small dataset. As a result, overfitting occurs even when using the improved training process (See Figure 5). Although the overall results are similar to the previous training, the training curves differ,

Decoding Strategies (temperature, top-k)

After training a language model, it can be used to generate text. However, the quality and diversity of the generated output depend not only on the model itself but also on how the next token is selected. The method used to choose the next token is called a **decoding strategy**.

The simplest approach is **Greedy Decoding** - selecting the token with the highest probability at each step. This may yield coherent text, but can also lead to repetitive or predictable outputs. To introduce controlled randomness, several decoding strategies are used. Two common techniques are **temperature scaling** and **top-k sampling**, which enable to balance coherence and diversity during generation.

Temperature Scaling

Temperature scaling alters the probability distribution before selecting the next token. Before applying the softmax function, the model's logits are divided by a temperature value. This controls how sharp or flat the probability distribution is going to be:

- **temperature = 1** → original distribution
- **temperature < 1** → sharper distribution (more deterministic output)
- **temperature > 1** → flatter distribution (more diverse output)

Higher temperatures mean more creativity but may also produce grammatically incorrect or nonsensical sentences.

Exercise 5.1

To study the influence of temperature, the probability distribution was computed for several temperature values: **0.01**, **0.1**, **1**, **4**, and **16**. For each temperature, the `print_sampled_tokens` function samples tokens 1000 times according to their probabilities and prints the frequency of each. A faster and more accurate way to determine how often the token *pizza* appears is to directly check its probability in the softmax distribution instead of relying on sampling.

```
temperatures = [0.01, 0.1, 1, 4, 16]
for t in temperatures:
    probas = softmax_with_temperature(next_token_logits, t)
    print(f"temperature: {t}")
    print_sampled_tokens(probas)
    print(f"probability of pizza: {probas[6]}")
    print("")
```

```
temperature: 0.01
0 x closer
0 x every
0 x effort
1000 x forward
probability of pizza: 0.0
```

... (complete result can be found in [2])

```
temperature: 16
128 x closer
109 x every
85 x effort
163 x forward
83 x inches
93 x moves
80 x pizza
148 x toward
111 x you
probability of pizza: 0.08631856739521027
```

The results show that the frequency of the token “pizza” increases as the temperature becomes larger:

Temperature	“pizza” samples (out of 1000)	Frequency of pizza (“pizza” samples / 1000)	Actual Probability of “pizza”
0.01	0	0	0.0
0.1	0	0	$\approx 2.97 \times 10^{-38}$
1	0	0	≈ 0.00010
4	23	0.023	≈ 0.0319
16	80	0.08	≈ 0.0863

For very low temperatures (0.01 and 0.1), the probability distribution becomes very sharp, so almost all probability is assigned to the most likely token (“forward”), and pizza is never sampled.

For higher temperatures (4 and 16), the distribution becomes flatter, increasing the chance of sampling less likely tokens such as pizza.

This behavior explains why very high temperatures often produce more creative but less coherent text, while very low temperatures lead to deterministic outputs.

Top-k Sampling

Another commonly used decoding strategy is **top-k sampling**. Instead of sampling from the entire vocabulary, the model selects the next token only from the k most probable tokens, all other tokens are ignored.

This prevents extremely unlikely tokens from being selected while still allowing some variation in the output.

Exercise 5.2 and 5.3

To explore the effects of temperature and top-k sampling, several combinations of these parameters were tested using both the locally trained model and the pretrained GPT-2 model. To obtain deterministic output, we can set **top_k = 1** and **temperature = 0**.

Model	temperature	top_k	Example generated output
Local	1.4	5	Every effort moves you know terrace. "I glanced after him, with a little last
Local	15	25	Every effort moves you as though, or Florence me afterward ins geniused that dep Mrs of such
Local	0	1	Every effort moves you?" "Yes--quite insensible to the irony. She wanted
GPT2-s	1.4	5	Every effort moves you forward in the life of the person, but it's a very hard thing
GPT2-s	15	25	Every effort moves you to reach where a particular value is most effective, and you try all these
GPT2-s	0	1	Every effort moves you forward. The first step is to understand the importance of your work

Experiments show that **temperature** and **top-k** strongly affect the randomness and structure of generated text.

With **top_k = 5**, the model samples only from the five most probable tokens, making more predictable and coherent sentences. This is helpful for tasks that require accuracy, such as technical writing, code generation, or question answering.

But, as **temperature = 1.4** and **top_k = 5**, the local model generated "*know terrace*" (incoherent), while GPT-2 produced "*forward in the life of...*" (coherent). This is because the local model was trained on only ~4,000 tokens and thus had a poor probability distribution.

If you increase **top_k to 25**, you can sample from a larger pool of tokens, so you will have more variability. A higher temperature (e.g. **15**) makes less probable tokens appear more often, resulting in more creative but less structured text, good for storytelling or brainstorming.

Eventually, generation is fully deterministic with temp=0 and top_k=1, always selecting the most likely token and always producing the same output.

Saving and Loading Model Weights

Training large language models is computationally expensive, so instead of retraining from scratch, trained models are usually stored and reused.

In PyTorch, model parameters are stored in the state dictionary (state_dict), which is a dictionary mapping layer names to their corresponding tensors. To save a model, one simply saves this state dictionary (usually in a .pth file), and later loads the dictionary into a model with the same architecture.

If you need to resume training, you should also save the optimizer state, because optimizers like Adam or AdamW keep track of additional information about past gradients, which allows you to pick up training where you left off.

Exercise 5.4

So, when we want to continue training the model in a new session, we need to load the previously saved checkpoint with the model and optimizer states.

First, the checkpoint is loaded and the model architecture is re-initialized with the same configuration. Then, the load_state_dict function is used to restore the saved model parameters.

```
checkpoint = torch.load("model_and_optimizer.pth", map_location=device)
model_imported = GPTModel(GPT_CONFIG_124M)
model_imported.load_state_dict(checkpoint["model_state_dict"])
model_imported.to(device)
```

Next, the optimizer is recreated and its state is restored from the checkpoint.

```
optimizer_imported = torch.optim.AdamW(
    model_imported.parameters(), lr=5e-4, weight_decay=0.1
)
optimizer_imported.load_state_dict(checkpoint["optimizer_state_dict"])
```

With the model and optimizer states restored, training can continue as usual. We use the function train_model_simple with num_epochs=1 to train the model for one more epoch.

```
model_imported.train()
num_epochs = 1
train_losses, val_losses, tokens_seen = train_model_simple(
    model_imported, train_loader, val_loader, optimizer_imported, device,
    num_epochs=num_epochs, eval_freq=5, eval_iter=5,
    start_context="Every effort moves you", tokenizer=tokenizer
)
```

Recall that the initial training loss at training step 0 was 9.821 and validation loss was 9.931. Following the first training process, these values were reduced to 0.708 and 6.387, respectively. When continuing training from the saved checkpoint (instead of starting from scratch), the first step already obtained a training loss of 0.550 and a validation loss of 6.417, indicating that the previously learned weights were successfully restored. Full implementation is given in [2] (file Chapter5).

That is a proof that we can save the model parameters and the optimizer state and resume training without losing what the model have learned so far.

Loading Pretrained GPT-2 Weights

Training large language models from scratch is computationally expensive, and thus pretrained models such as **GPT-2** are often reused instead of training new models from the beginning. Pretrained weights can be loaded and used directly for text generation or further adapted through fine-tuning.

OpenAI released several GPT-2 variants that differ mainly in the number of parameters. The downloaded parameters can be mapped to the corresponding layers of the implemented GPT model, including token embeddings, positional embeddings, attention projections, feed-forward layers, and normalization parameters.

Exercise 5.5

To evaluate the pretrained GPT-2 model on *The Verdict* dataset, the training and validation losses were calculated using the `calc_loss_loader` function.

Training loss: 3.5319405794143677 Validation loss: 3.5318045616149902

The results here suggest that the pretrained GPT-2 model does a reasonably good job on *The Verdict* dataset. However, it is possible that this text was included in the data used to pretrain GPT-2. If so, the results would not fully reflect the model's ability to generalize to previously unseen text.

To investigate the model further, another experiment was performed on a different dataset. This dataset consists of recent English translations of Georgian history notes, produced by Giorgi Gogsadze, a co-author of this report. Since these notes were created after GPT-2 was trained, they could not have been part of its pretraining data. The obtained loss for this dataset was: **2.895763317743937**

The comparable loss values suggest that the pretrained GPT-2 model captures general language patterns rather than relying solely on memorized training data. The implementation of this experiment can be found in [2] (GPT2 file).

Exercise 5.6

To explore the effect of model size on text generation, the **GPT-2 small model (124M parameters)** and the **GPT-2 XL model (1558M parameters)** were tested using the same prompts. Two example prompts were used:

Prompt 1: *“Every effort moves you”*

- **GPT-2 small:**

“Every effort moves you as far as the hand can go until the end of your turn unless something happens. This would remove you from a battle.”

- **GPT-2 XL:**

“Every effort moves you further to where you do not actually want to grow; you are still too much of an outsider. This has been demonstrated clearly.”

Prompt 2: *“The Capital of Georgia is”*

- **GPT-2 small:**

“The Capital of Georgia is the place the capital can build infrastructure like a rail network. It's a very popular destination right after Georgia's economic prosperity.”

- **GPT-2 XL:**

“The Capital of Georgia is the place the capital can use because it was founded within Georgian territory but does not belong to it.”

The examples above suggest that the larger GPT-2 XL model tends to generate more fluent and stylistically coherent sentences, while the smaller model sometimes produces less structured or logically inconsistent text.

This is expected behavior . Bigger models have more parameters , which means they can capture more complex language patterns and relationships between words . Larger models tend to produce better quality and more natural text, but they require much more computational power at the same time.

References

- [1] Raschka, S. (2024). *Build a large language model (from scratch)*. Shelter Island, NY: Manning Publications.
- [2] Giorgi Gogsadze, Revaz Goguadze. LLM from Scratch Code Repository. Available at: <https://github.com/GiorgiGogsadze/LLM-From-Scratch-Thesis>
- [3] Loshchilov, I., & Hutter, F. (2017). Decoupled Weight Decay Regularization. arXiv:1711.05101.